

程序设计实践

The Practice of Programming

性能(Performance)

性能

时间相关

- 程序执行的速度（响应时间）
- 大量并发时，对**CPU**资源的占用

空间相关

- 对内存的占用
- 对磁盘的占用
- 对网络带宽的占用

是否应该进行性能优化？

成本

- 性能优化本身需要时间成本。
- 性能优化对程序的修改可能会引入BUG。
- 性能优化可能会导致代码更加复杂晦涩，可读性变差。
- 性能优化可能会导致程序的可移植性变差。
- ...

收益

- 性能优化可以让用户体验更好，同类产品中更有竞争力。
- 性能优化之后，如果提供同样的服务，可以使用更少的硬件，节省费用，节约能源，低碳环保。
- ...
- 但是，所有这些收益的大小都依赖于这个程序的用户数量。

Python 3.11.0

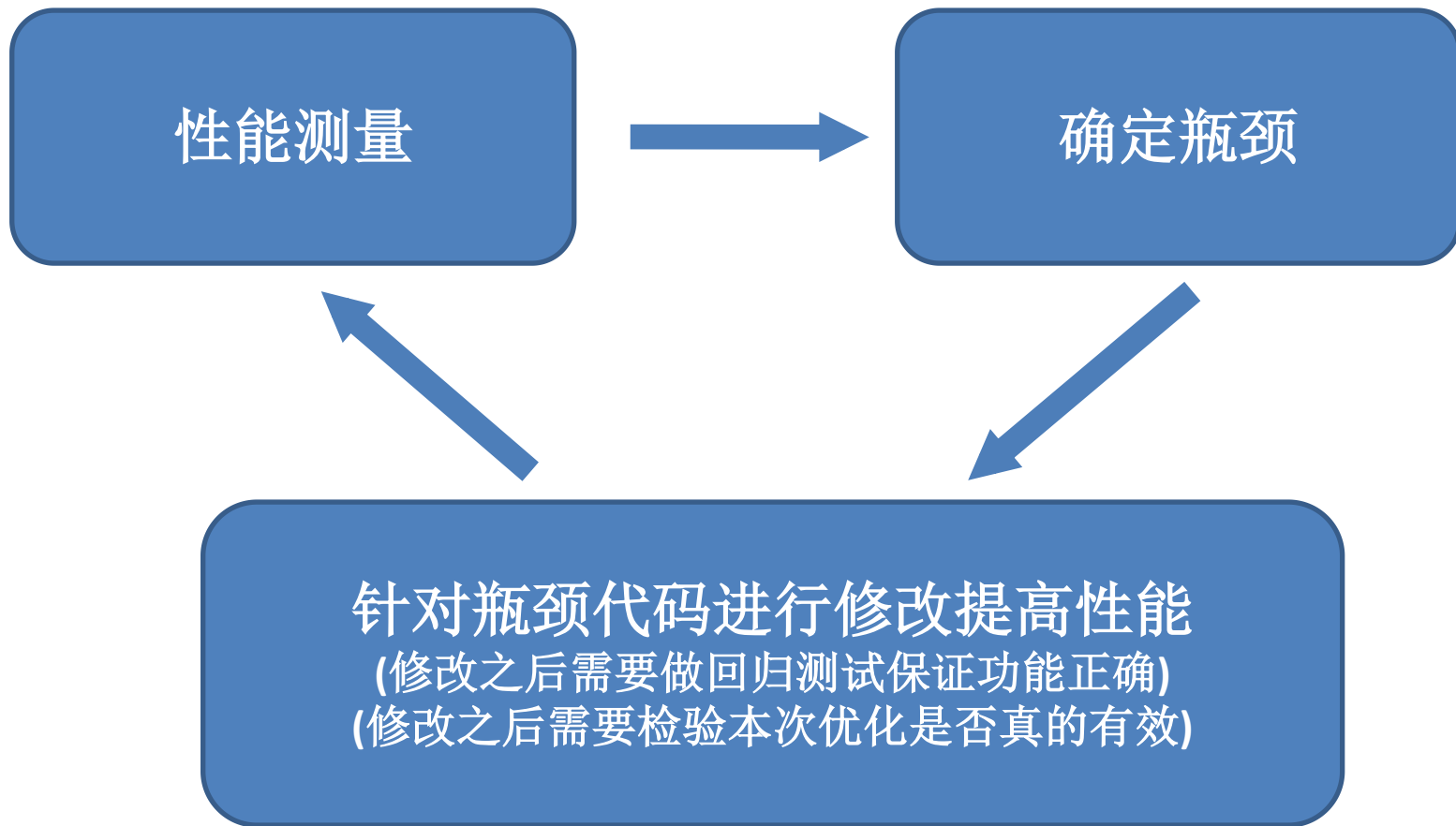
Release Date: Oct. 24, 2022



This is the stable release of Python 3.11.0

Operation	Form	Specialization	Operation speedup (up to)	Contributor(s)
Binary operations	<code>x+x</code> ; <code>x*x</code> ; <code>x-x</code> ;	Binary add, multiply and subtract for common types such as <code>int</code> , <code>float</code> , and <code>str</code> take custom fast paths for their underlying types.	10%	Mark Shannon, Dong-hee Na, Brandt Bucher, Dennis Sweeney
Subscript	<code>a[i]</code>	Subscripting container types such as <code>list</code> , <code>tuple</code> and <code>dict</code> directly index the underlying data structures. Subscripting custom <code>__getitem__</code> is also inlined similar to Inlined Python function calls .	10-25%	Irit Katriel, Mark Shannon
Store subscript	<code>a[i] = z</code>	Similar to subscripting specialization above.	10-25%	Dennis Sweeney
Calls	<code>f(arg)</code> <code>C(arg)</code>	Calls to common builtin (C) functions and types such as <code>len</code> and <code>str</code> directly call their underlying C version. This avoids going through the internal calling convention.	20%	Mark Shannon, Ken Jin
Load global variable	<code>print len</code>	The object's index in the globals/builtins namespace is cached. Loading globals and builtins require zero namespace lookups.	[1]	Mark Shannon
Load attribute	<code>o.attr</code>	Similar to loading global variables. The attribute's index inside the class/object's namespace is cached. In most cases, attribute loading will require zero namespace lookups.	[2]	Mark Shannon
Load methods for call	<code>o.meth()</code>	The actual address of the method is cached. Method loading now has no namespace lookups – even for classes with long inheritance chains.	10-20%	Ken Jin, Mark Shannon
Store attribute	<code>o.attr = z</code>	Similar to load attribute optimization.	2% in pyperformance	Mark Shannon
Unpack Sequence	<code>*seq</code>	Specialized for common containers such as <code>list</code> and <code>tuple</code> . Avoids internal calling convention.	8%	Brandt Bucher

性能优化的一般过程:



性能测量: **time**

```
$ time gzip UNI.INFO.MP.3
```

```
real      0m5.800s
```

```
user      0m5.762s
```

```
sys       0m0.034s
```

```
$ time sleep 10
```

```
real      0m10.001s
```

```
user      0m0.000s
```

```
sys       0m0.000s
```

real: 程序开始到结束，现实世界经过的时间

user: 程序在用户态运行所消耗的CPU时间

sys: 程序在内核态运行所消耗的CPU时间

性能测量: top

对于长时间持续提供服务的程序来说, 无法使用**time**测量其运行时间。
可以根据当前的请求频率和并发数量配合**top**命令查看CPU占用。

```
top - 16:30:24 up 31 days, 6:41, 3 users, load average: 0.99, 0.73, 0.43
Tasks: 280 total, 3 running, 275 sleeping, 0 stopped, 2 zombie
Cpu0  :  5.0%us,  7.0%sy,  0.0%ni, 86.3%id,  0.0%wa,  0.0%hi,  1.7%si,  0.0%st
Cpu1  : 38.1%us,  2.7%sy,  0.0%ni, 59.2%id,  0.0%wa,  0.0%hi,  0.0%si,  0.0%st
Cpu2  : 19.1%us,  1.3%sy,  0.0%ni, 79.3%id,  0.0%wa,  0.0%hi,  0.3%si,  0.0%st
Cpu3  : 63.7%us,  6.3%sy,  0.0%ni, 24.7%id,  0.0%wa,  0.0%hi,  5.3%si,  0.0%st
Cpu4  :  6.6%us,  0.7%sy,  0.0%ni, 92.7%id,  0.0%wa,  0.0%hi,  0.0%si,  0.0%st
Cpu5  :  1.7%us,  0.3%sy,  0.0%ni, 98.0%id,  0.0%wa,  0.0%hi,  0.0%si,  0.0%st
Cpu6  :  0.7%us,  0.3%sy,  0.0%ni, 99.0%id,  0.0%wa,  0.0%hi,  0.0%si,  0.0%st
Cpu7  :  9.0%us,  0.3%sy,  0.0%ni, 90.3%id,  0.0%wa,  0.0%hi,  0.3%si,  0.0%st
Mem: 16084392k total, 7837000k used, 8247392k free, 450484k buffers
Swap: 8110076k total, 295516k used, 7814560k free, 2838932k cached
```

PID	USER	PR	NI	VIRT	RES	SHR	S	%CPU	%MEM	TIME+	COMMAND
98983	proxy	20	0	936m	874m	4124	R	75.4	5.6	5:34.68	uniframe
98981	proxy	20	0	61128	5252	2636	S	10.6	0.0	0:48.35	uniframe
98987	proxy	20	0	357m	294m	3032	S	8.6	1.9	0:33.55	uniframe
98991	proxy	20	0	357m	294m	3032	S	8.6	1.9	0:33.33	uniframe
98992	proxy	20	0	356m	294m	3032	R	8.6	1.9	0:33.25	uniframe
98984	proxy	20	0	357m	294m	3032	S	8.3	1.9	0:33.26	uniframe
98985	proxy	20	0	357m	294m	3032	S	8.3	1.9	0:33.38	uniframe
98988	proxy	20	0	357m	294m	3032	S	8.3	1.9	0:33.53	uniframe
98990	proxy	20	0	356m	293m	3032	S	8.3	1.9	0:33.40	uniframe
98993	proxy	20	0	356m	294m	3032	S	8.0	1.9	0:33.43	uniframe
98986	proxy	20	0	357m	294m	3032	S	7.6	1.9	0:33.47	uniframe
98989	proxy	20	0	357m	294m	3032	S	7.6	1.9	0:33.20	uniframe
98980	proxy	20	0	12704	916	668	S	0.3	0.0	0:00.06	sc0
99107	proxy	20	0	21244	1680	1368	S	0.3	0.0	0:00.41	sclist

左图: 一个sip-proxy系统, 在每秒500次呼叫压力下的性能测试期间, 使用top命令显示的结果。

性能测量: gprof

```
$ g++ -pg test.C
```

使用-pg选项编译，生成携带gprof统计功能的可执行文件a.out

```
$ ./a.out
```

执行a.out结束后，会生成gmon.out（统计）文件

```
$ gprof a.out gmon.out
```

使用gprof读取gmon.out，可以生成针对这个程序各个函数的性能测量结果，包括：

- 程序中每个函数被调用的次数，执行的累计时间，时间占用的百分比
- 程序中每个函数之间的调用关系，被谁调用（调用次数，时间占用），自己调用了谁（调用次数，时间占用）

性能测量: gprof

Flat profile:

Each sample counts as 0.01 seconds.

% time	cumulative seconds	self seconds	calls	self s/call	total s/call	name
10.12	8.42	8.42	894222	0.00	0.00	resip::MsgHeaderScanner::scanChunk(char*, unsigned int,
1.70	9.84	1.42	43105822	0.00	0.00	resip::operator<(resip::Timer const&, resip::Timer const&
1.26	10.89	1.05	27717808	0.00	0.00	resip::Data::Data(resip::Data const&)
1.18	11.87	0.98	122360401	0.00	0.00	resip::Data::~~Data()
1.14	12.82	0.95	5426787	0.00	0.00	resip::Data::rawHash(unsigned char const*, unsigned lon
1.13	13.76	0.94	14063644	0.00	0.00	resip::HeaderFieldValueList::~~HeaderFieldValueList()
1.12	14.69	0.94	1994740	0.00	0.00	resip::SipMessage::cleanUp()
0.94	15.47	0.78	14856700	0.00	0.00	std::vector<resip::HeaderFieldValue*, std::allocator<re
0.85	16.18	0.71	76712446	0.00	0.00	erFieldValue**, std::vector<resip::HeaderFieldValue*, std::allocator<resip::HeaderFieldValue* > >, resip::He
0.82	16.86	0.68	16458652	0.00	0.00	resip::Data::Data()
0.68	17.43	0.57	142616640	0.00	0.00	resip::ParserCategory::clear()
> >::__normal_iterator(resip::HeaderFieldValue** const&)						__gnu_cxx::__normal_iterator<resip::HeaderFieldValue*
0.64	17.96	0.53	3296669	0.00	0.00	resip::TransactionState::process(resip::TransactionCon
0.62	18.48	0.52	38183741	0.00	0.00	resip::ParseBuffer::skipChar()
0.62	19.00	0.52	7864860	0.00	0.00	resip::ParseBuffer::skipToOneOf(char const*)

index %time self children called name

		8.42	6.60	894222/894222	resip::UdpTransport::process(CFdSet&) [12]
[14]	18.0	8.42	6.60	894222	resip::MsgHeaderScanner::scanChunk(char*, unsigned int, char**) [14]
		0.10	4.64	10259632/12359632	resip::processMsgHeaderFieldNameAndValue(resip::SipMessage*, int
		0.01	0.97	894222/994222	resip::processMsgHeaderStatusLine(resip::SipMessage*, char*, unsig
		0.07	0.81	10259632/12359632	resip::lookupMsgHeaderFieldInfo(char*, unsigned int*, int*, bool

确定瓶颈 - CPU 占用瓶颈

性能优化，首先应该找到性能的瓶颈在哪里，不解决瓶颈的性能，其它部分的优化是没有意义的。

用 **top** 测量，可以看到每个进程的 **cpu** 占用，我们根据这个信息可以基本判断瓶颈在哪个进程。

用 **gprof** 测量，可以看到每个函数 **cpu** 占用，我们根据这个信息可以基本判断瓶颈在哪个函数。

为什么让我们等这么久？

对不起，我很忙。



瓶颈君1

确定瓶颈 - 同步等待瓶颈

如果是同步等待导致的瓶颈，用top和gprof无法直接看出瓶颈所在。可以通过代码隔离的方式寻找瓶颈。



为什么让我们等这么久？

对不起，我在等指示。



瓶颈君2

时间性能优化

使用编译器优化选项

并发设计

I/O管理的优化

使用更好的逻辑设计

使用更好的算法和数据结构

其它优化技巧:

- 预先计算某些值
- 避免不必要的参数复制
- 避免不必要的堆空间分配
- 避免大量的if/else分支
- 写个程序亲自检验某些操作的性能, 决定是否使用
- ...

时间性能优化 - 使用编译器的优化选项

使用编译器的优化选项是最容易的一件事，几乎不必做任何改动。例如使用gcc编译，只要在编译命令里加上-O3即可。这也是首先应该考虑的方法。

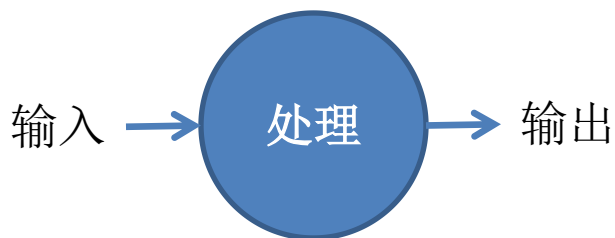
编译优化之后，需要做严格的回归测试。因为对于复杂的程序，编译优化有一定的可能性会改变程序的一些行为。

编译优化是对所有代码做的无差别的优化，针对的是代码细节，并不知道你的瓶颈在哪里。它也许能将性能提高一两成，或者提高一倍，但不会有数量级上的改变。特别是针对一些已经在编程中注意了性能细节的程序，编译优化的作用非常有限。

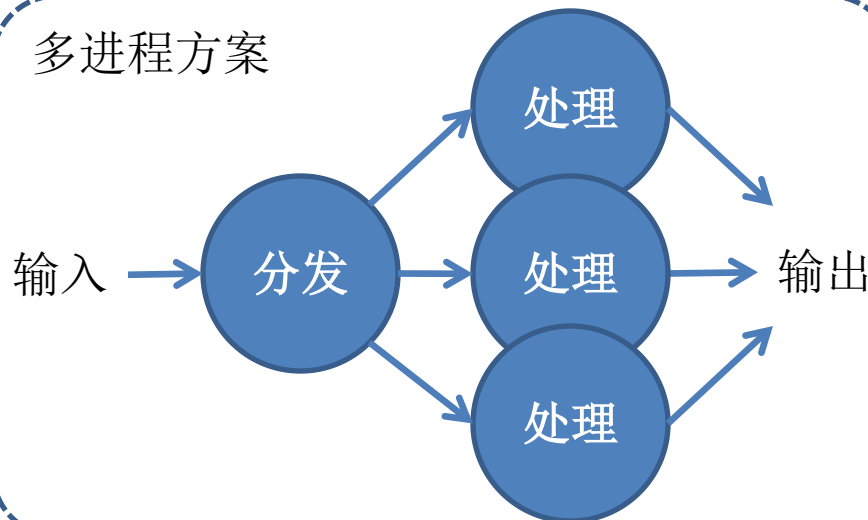
再说，很有可能，不幸的是，这个程序的前任程序员早已开启了编译优化。

时间性能优化 - 并发设计

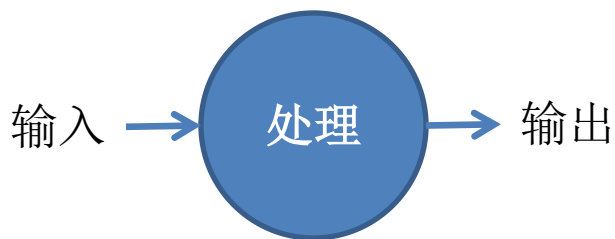
单进程方案



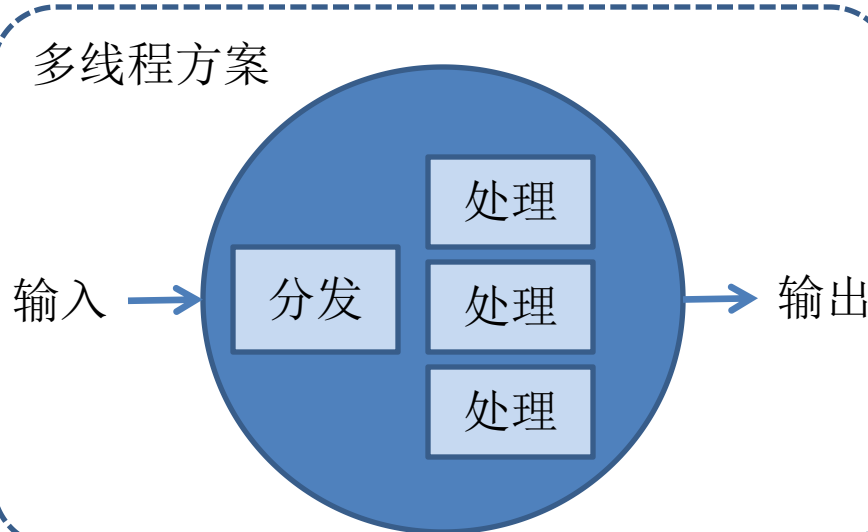
多进程方案



单线程方案

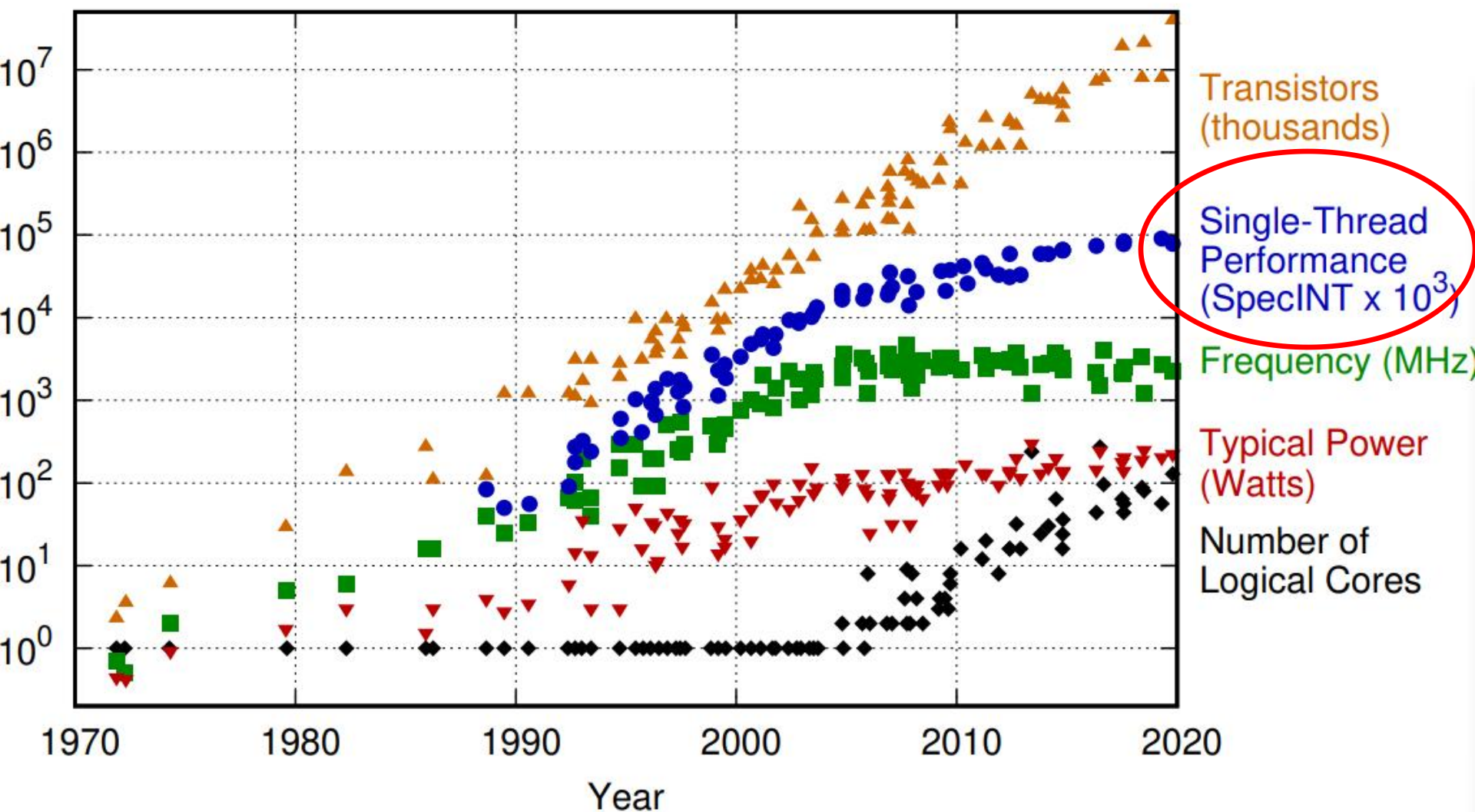


多线程方案



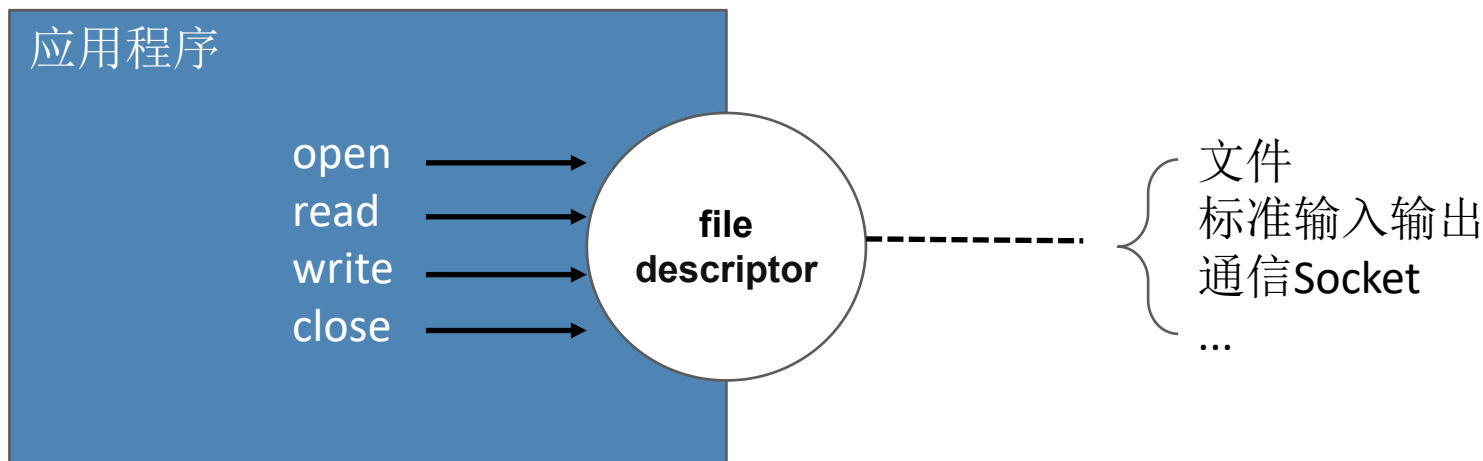
对比维度	多进程	多线程	总结
数据共享、同步	数据共享复杂，需要用IPC；数据是分开的，同步简单	因为共享进程数据，数据共享简单，但也是因为这个原因导致同步复杂	各有优势
内存、CPU	占用内存多，切换复杂，CPU利用率低	占用内存少，切换简单，CPU利用率高	线程占优
创建销毁、切换	创建销毁、切换复杂，速度慢	创建销毁、切换简单，速度很快	线程占优
编程、调试	编程简单，调试简单	编程复杂，调试复杂	进程占优
可靠性	进程间不会互相影响	一个线程挂掉将导致整个进程挂掉	进程占优
分布式	适应于多核、多机分布式；如果一台机器不够，扩展到多台机器比较简单	适应于多核分布式	进程占优

48 Years of Microprocessor Trend Data



Original data up to the year 2010 collected and plotted by M. Horowitz, F. Labonte, O. Shacham, K. Olukotun, L. Hammond, and C. Batten
New plot and data collected for 2010-2019 by K. Rupp

时间性能优化 - I/O管理的优化

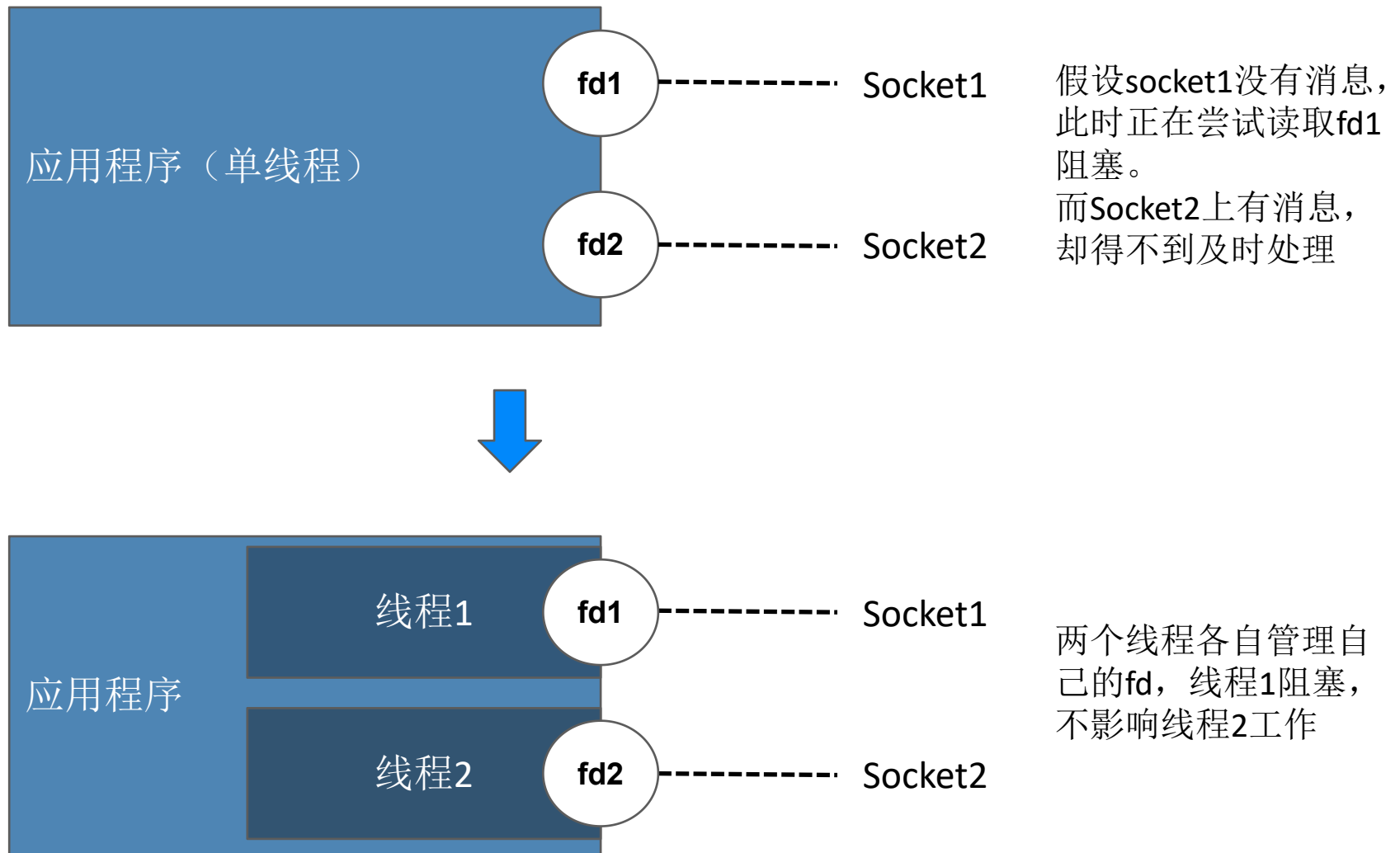


一般而言，应用程序通过文件描述符（**file descriptor**，简称**fd**）来访问I/O，**open**用来获取**fd**，**close**用来关闭**fd**，**read**用来读取输入，**write**用来输出。

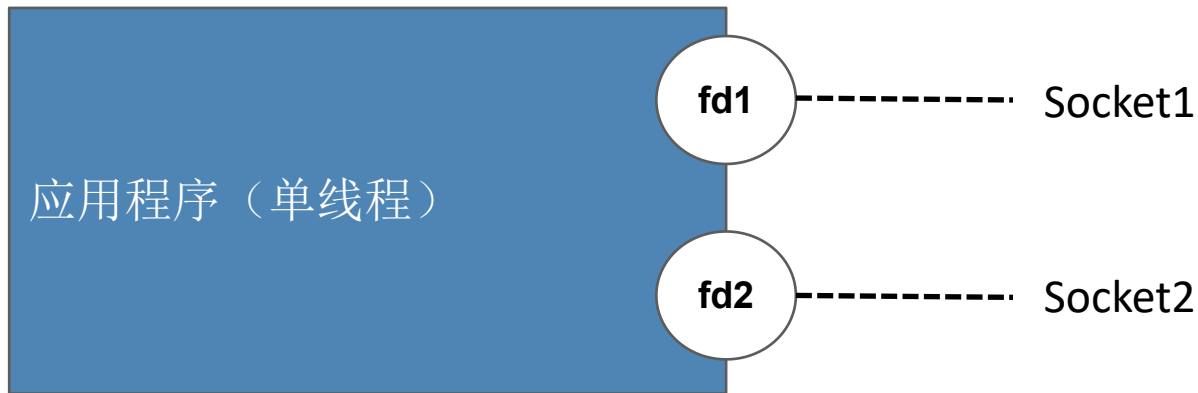
当前**fd**不可写的时候，进行**write**，或者当前**fd**不可读的时候进行**read**，都会导致进程阻塞（不占用CPU，但是应用程序当前线程不能做别的事情）。

如果性能测试结果差，同时CPU有很大的空闲，那么很大可能是这个程序的I/O管理设计有问题。

时间性能优化 - I/O管理的优化(多线程)



时间性能优化 - I/O管理的优化(select)



```
fd_set fdset; // 定义一个fd集合
FD_ZERO(&fdset); // 将集合初始化为0
FD_SET(fd1, &fdset); // 将fd1加入集合。
FD_SET(fd2, &fdset); // 将fd2加入集合。线程内的所有fd都必须加入到一个fd集合中，不能遗漏
int maxfd = fd1 > fd2 ? fd1 : fd2; // 求fd1和fd2的最大值
```

```
timeval waittime; // 定义一个等待时间，设置等待时间为10毫秒
waittime.tv_sec = 0;
waittime.tv_usec = 10000;
```

```
// 对fd集合进行可读select，最大等待10毫秒，10毫秒之内集合中的任何一个fd可读，则select会立即退出
if(select(maxfd + 1, &fdset, NULL, NULL, &waittime) <= 0)
    return; // 如果返回<=0，表示等待10毫秒之后，没有任何一个fd可读。
if(FD_ISSET(fd1, &fdset)) // 如果fd1可读，处理fd1
    read(fd1, ...);
if(FD_ISSET(fd2, &fdset)) // 如果fd2可读，处理fd2
    read(fd2, ...);
```

时间性能优化 - I/O管理的优化(epoll)

如果需要管理大量的fd，例如数千个，甚至数万个，则应该使用epoll代替select。这种情况多线程不能解决问题。

epoll和select的实现机制类似，都是将所有fd加入集合一起管理，使得活跃fd尽快得到处理，避免不同fd之间的等待。epoll比select有如下优点：

- 内核参数限制了select的fd集合的最大数量(默认1024)。epoll则没有最大fd个数限制。
- 因为select每次会对fd集合全部扫描，所以随着fd数量的增加，select的效率必然降低。而epoll则是事件驱动调用回调函数实现，不会因fd数量增加导致性能降低。

I/O管理的优化

- 线程内是否多个fd各自阻塞，没有统一管理？
- 能否使用多线程，确保每个线程只有一个fd？
- 如果用了select管理了多个fd，检查一下是否有遗漏？
- 如果fd数量巨大，是否应该考虑改为epoll实现？

时间性能优化 - 使用更好的逻辑设计

如果能从一个逻辑中去掉一个步骤，远比优化它效果要好。

梳理一下整个业务逻辑，看看哪些是不必要的？哪些是重复的操作？去掉它们，让逻辑更简洁的同时，性能更好。

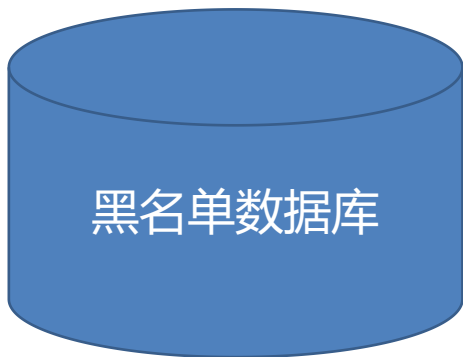
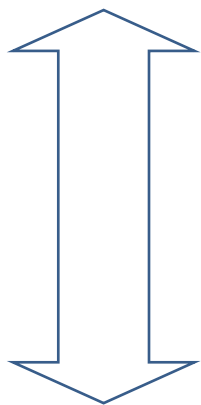


**Design/
Code
Review**

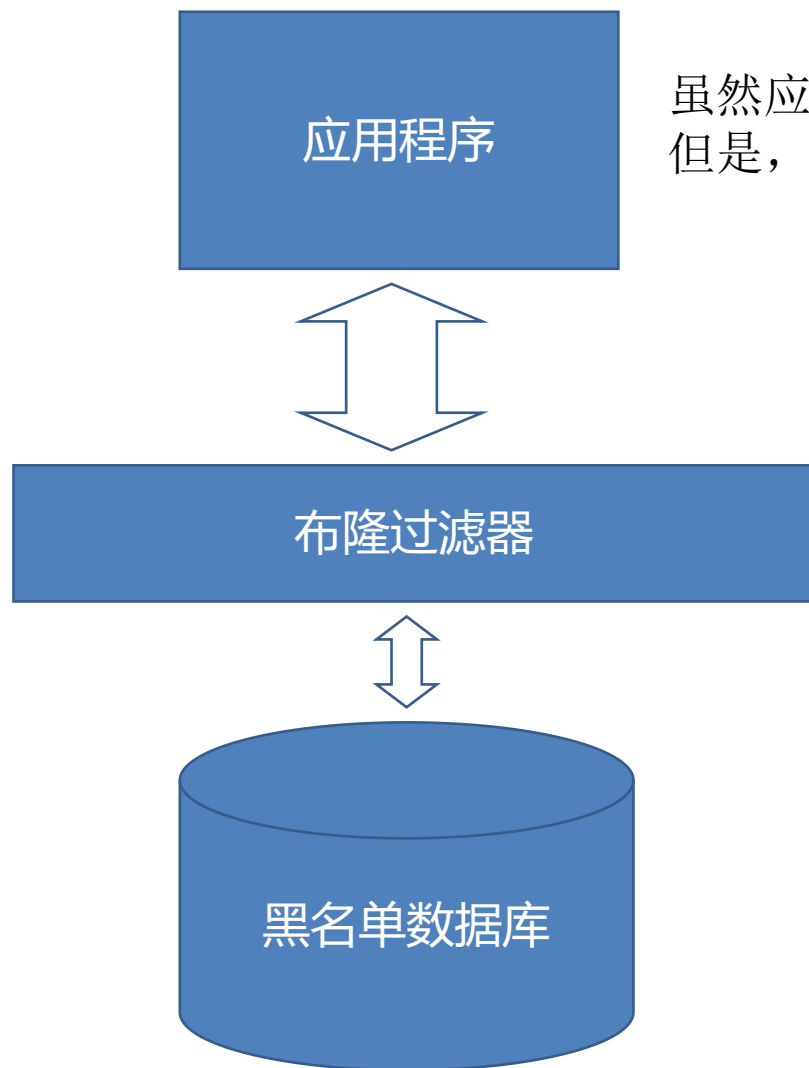
时间性能优化 - 使用过滤器



虽然应用需要对黑名单数据库进行大量查询，
但是，大部分都是查不到情况



时间性能优化 - 使用过滤器



虽然应用需要对黑名单数据库进行大量查询，但是，大部分都是查不到情况

布隆过滤器 BF (Bloom Filter)

- 1970年Burton Bloom在论文《Space/time trade-offs in hash coding with errors》中提出
- 用一系列随机映射函数 + 一个很长的二进制向量实现，用准确率换空间
- 空间效率和查询时间都远超过一般的算法
- 能够准确判断一个元素不在一个集合中
- 基本准确判断一个元素在一个集合中。

时间性能优化 - 使用更好的算法和数据结构

如果前面所述的改进方法不能满足性能需求，在找到瓶颈函数之后，可以进一步考虑该函数是否使用了合适的算法和数据结构？例如，可能之前数据规模很小，使用了顺序查找，但是随着需求的变化，数据规模越来越大，可能就需要建立一个**HASH**表来存储数据了。

如果已经使用**HASH**表来存储数据了，查一下这个**HASH**函数是否合理？可能因为索引取值范围的变化，以前的**HASH**函数会导致碰撞率非常高。特别是对字符串的**HASH**函数，例如为了简便，以前直接取字符串的前4个字节转换为整数再取模作为数组下标，但是随着需求的变化，现在所有的字符串都有个相同的前缀！

散列数组是否足够大？是否和当前的数据量相符合？是否有必要改为散列数组随着数据量增大自动增加的模式？

如果存储条件允许，是否可以改为“直接数组下标访问”模式？

时间性能优化 - 预先计算某些值

例如，大量的坐标要从一个坐标系的坐标值(x' , y')转换为另一个坐标系的坐标值(x , y)，使用下面的公式（其中角 θ 是两个坐标系的夹角）：

$$\begin{aligned}x &= x' \cos \theta - y' \sin \theta \\ y &= y' \cos \theta + x' \sin \theta\end{aligned}$$

```
// 输入x1和y1以及坐标系夹角angle，输出变换后的x, y
void coord_trans(float x1, float y1, float& x, float& y, float angle){
    x = x1 * cos(angle) - y1 * sin(angle);
    y = y1 * cos(angle) + x1 * sin(angle);
}
```

```
class CoordTransfer{
    float sin_angle, cos_angle;
public:
    CoordTransfer(float angle){ // 一次调用
        sin_angle = sin(angle); // 预先计算sin(angle)
        cos_angle = cos(angle); // 预先计算cos(angle)
    }
    void compute(float x1, float y1, float& x, float& y){ // 大量调用，直接使用预先计算的值
        x = x1 * cos_angle - y1 * sin_angle;
        y = y1 * cos_angle + x1 * sin_angle;
    }
};
```

时间性能优化 - 延后计算某些值

为什么有时候需要预先计算某些值，又有时候需要延后计算某些值？

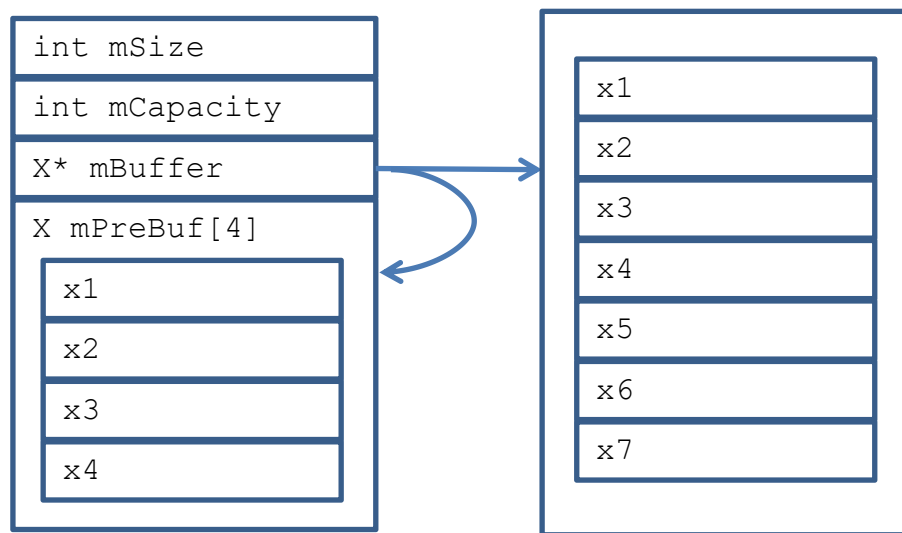
判断准则在于，这些值到底会多么频繁地被使用：

- 如果同样的值频繁使用，则要预先计算。
- 如果经常变化的值，偶尔使用，则要延后计算，使用的时候再计算。

```
bool proc_msg(const char* msg, int length){
    std::string hex_str;
    msg_to_hex(hex_str, msg, length); // 将不可打印的msg以十六进制形式转换到hex_str中
    if (xx错误){
        error_log("xx错误, 消息: %s\n", hex_str.c_str());
        return false;
    }
    // 其它正常处理...
}
```

```
bool proc_msg(const char* msg, int length){
    if (xx错误){
        // hex_str在正常流程中不会用到，只有在发生错误的时候才会用到
        std::string hex_str;
        msg_to_hex(hex_str, msg, length); // 将不可打印的msg以十六进制形式转换到hex_str中
        error_log("xx错误, 消息: %s\n", hex_str.c_str());
        return false;
    }
    // 其它正常处理...
}
```

时间性能优化 - 统计数据结构实际使用情况



自增数组的mPreBuf应该定义多大？

- 太大，浪费空间
- 太小，会频繁resize，浪费时间
- 统计应用的实际情况决定

思考题：

如果程序中有一个链表，但是99%的情况，这个链表仅有一项，该如何优化？

时间性能优化 - 避免不必要的堆内存分配

```
struct tm* curTm = new struct tm;  
localtime_r(&curTime, curTm);  
// ... 对curTm进行处理  
delete curTm;
```

```
struct tm curTm;  
localtime_r(&curTime, &curTm);  
// ... 对curTm进行处理
```

```
char* buffer = new char[BUFFER_SIZE];  
read(fd, buffer, BUFFER_SIZE);  
// ... 对buffer进行处理  
delete buffer;
```

```
char buffer[BUFFER_SIZE];  
read(fd, buffer, sizeof(buffer));  
// ... 对buffer进行处理
```

堆的实现是有一点复杂的，需要管理索引，需要管理碎片等等。如果一个程序使用了太多的堆里面的内存分配，必然是影响效率的。对于仅在一个函数内部局部使用的内存块，直接在栈里分配是最好的。

时间性能优化 - 避免不必要的参数复制

```
struct A{  
    ...  
};  
void foo(struct A a)  
{  
    ...  
}
```

```
void foo(const struct A& a)  
{  
    ...  
}
```

```
void foo(struct A* a)  
{  
    ...  
}
```

时间性能优化 - 避免不必要的多次查询

```
struct Data
{
    int a;
    int b;
};
std::map<int, Data*> table;
```

```
bool set_data(int i, int a, int b)
{
    if (table.find(i) == table.end())
        return false;
    table[i]->a = a;
    table[i]->b = b;
    return true;
}
```

```
bool set_data(int i, int a, int b)
{
    auto iter = table.find(i);
    if (iter == table.end())
        return false;
    iter->second->a = a;
    iter->second->b = b;
    return true;
}
```

需要对map, hashtable等容器中的某个元素多次操作的时候, 第一次查询获得其指针、引用或者迭代器(iterator), 可以保存好, 后续操作的时候直接使用它们, 不要每次都再查询一遍。

时间性能优化 - 避免大量的if/else

如果一个功能，需要判断一个变量的值，不同的值做不同的事情。如果这个分支数量特别大，可以考虑优化它，方法有两个：

- 如果被判断的变量是整数，可以考虑改为switch/case实现。因为大部分编译器都会对switch/case做优化，使用直接下标访问，或者hash表来进行查询
- 可以将这些“不同的事情”实现为原型相同的函数，创建一个取值和函数指针对应的hash表。

```
int foo(std::string& name, int x)
{
    if(name == "aaa"){
        ...
        return ...;
    }
    if(name == "bbb"){
        ...
        return ...;
    }
    if(name == "ccc") {
        ...
        return ...;
    }
    ...
    else
        return -1;
}
```

```
int foo_aaa(int x) { ... return ...; }
int foo_bbb(int x) { ... return ...; }
int foo_ccc(int x) { ... return ...; }
...
typedef int FOO(int x);
std::map<std::string, FOO*> foo_map;

foo_map["aaa"] = foo_aaa;
foo_map["bbb"] = foo_bbb;
foo_map["ccc"] = foo_ccc;
...
int foo(std::string& name, int x){
    std::map<std::string, FOO*>::iterator i
        = foo_map.find(name);
    if(i == foo_map.end()){
        return -1;
    }
    return i->second(x);
}
```

时间性能优化 - 写个小程序亲自检验

```
#include <stdio.h>
int main() {
    int x = 123456789;
    int a = 1999999;
    int b = 1999998;
    for(int i = 0; i < 100000000; i++) {
        x = x * a;
        x = x / b;
    }
    printf("%d\n", x);
    return 0;
}
```

39

```
real    0m2.262s
user    0m2.260s
sys     0m0.000s
```

```
#include <stdio.h>
int main() {
    float x = 123456789.0;
    float a = 1999999.0;
    float b = 1999998.0;
    for(int i = 0; i < 100000000; i++) {
        x = x * a;
        x = x / b;
    }
    printf("%f\n", x);
    return 0;
}
```

504914971001240110762435805184.000000

```
real    0m1.534s
user    0m1.533s
sys     0m0.000s
```

随着时代的进步，多年前认为正确的结论也许已经不正确了。同样的程序，不同的环境也可能表现出不同的性能。对于拿不准的事情，最好写个小程序检验一下。

时间性能优化 - 一般**不要**使用这些小技巧

把简单循环展开（代码变长，出错概率增加。编译优化会自动做这个事情）

去掉函数调用，直接将函数代码展开（代码可读性重用性变差。编译优化会自动做这个事情）

用加法或者移位来代替乘法（不直观，代码可读性变差。编译优化会自动做这个事情）

浮点运算转换为整数运算（计算结果不够准确，代码可读性变差。现在的浮点运算处理器已经非常快，速度甚至和整数运算不相上下）

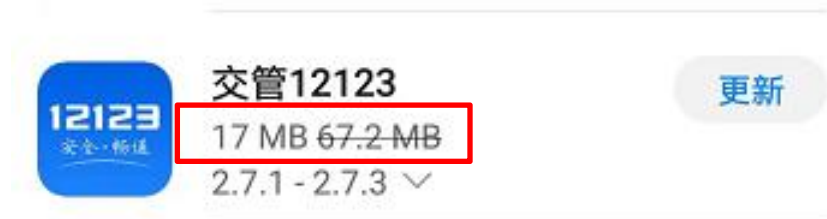
不可避免，随着编译优化以及硬件技术的不断提高，越来越多的“优化技巧”将会被抛弃。

空间性能优化

随着存储空间的成本越来越低，各种设备内存配置越来越大，大部分应用程序似乎已经不必小心翼翼地刻意节约内存空间。

但是，如果所有的程序员都肆意挥霍存储...

实际上，用户更愿意看到这样的信息...



空间性能优化 - 时间换空间

如果要表示一个稀疏的大数组（数组的大部分元素取值都为0）

```
int a[100000000] = {0};  
void init_a()  
{  
    a[1000] = 123;  
    a[3456] = 456;  
    a[12345678] = 789;  
    ...  
}
```

```
std::map<int, int> a;  
void init_a()  
{  
    a[1000] = 123;  
    a[3456] = 456;  
    a[12345678] = 789;  
    ...  
}
```

使用hash表来存储稀疏数组，占用空间会小得多，但是要牺牲一点时间性能。

空间性能优化 - 使用尽可能小的数据类型

如果要表示一个长度位100,000,000的bool数组

```
#define ARRAY_LEN 100000000  
bool a[ARRAY_LEN]; // 占用100,000,000字节
```

```
#define ARRAY_LEN 100000000  
#define UNIT_BITS 32  
unsigned int a[ARRAY_LEN / UNIT_BITS]; // 占用12,500,000字节  
inline bool get_bool(int i)  
{  
    if(i < 0 || i > ARRAY_LEN)  
        return false;  
    return a[i / UNIT_BITS] & (1 << (i % UNIT_BITS));  
}
```

使用一位表示一个bool值，存储空间仅需要1/8。

空间性能优化 - 节约网络带宽的几个例子

经典的语音压缩编码：16位的语音采样，压缩为8位的G711编码。

ASN.1的整数类型，采用最短补码表示：

- 0编码为00 (~~00 00 00~~ 00)
- 127编码为7F (~~00 00 00~~ 7F)
- 128编码位00 80 (~~00 00~~ 00 80)
- -1编码为FF (~~FF FF FF~~ FF)
- -128的编码为80 (~~FF FF FF~~ 80)
- -129的编码为FF 7F (~~FF FF~~ FF 7F)

ProtoBuf中的**sint32**和**sint64**类型，采的用**ZigZag**编码：

- 0编码为00
- -1编码为01
- 1编码为02
- -2编码为03
- 2编码为04
- ...

应用于短信的**7bit**编码。精选了拉丁字母，欧洲常用字母，常用符号等127个，7位表示一个符号，这样140字节可以表示160个符号。

小实验作业“程序性能优化”

本周四<http://10.112.112.240:8081/2010813434/ppt>下的“小作业3”中会发布一个存在性能问题的程序 `wordst_bad.cpp`。

这个程序的功能是对英文文本单词出现频率的统计。输入参数为要统计的文件名，输出统计结果（每个单词的出现次数以及行号，如果出现次数较多，只显示前20次出现的行号）。

要求如下：

- 测试当前给出的 `wordst_bad.cpp`，给出这些数据：
 - 测试用的文本的单词数量
 - 文本所含的不同单词的数量（根据运行结果得出）
 - 排名前三的单词及其数量
 - 执行所用时间（例如 `time` 命令的显示结果）
- 尝试找出程序中的性能问题，并加以改进，给出：
 - 改进之后的源程序 `wordst_good.cpp`
 - 修改的依据
 - 与 `wordst_bad` 相同测试数据下的执行时间比较（例如 `time` 命令的显示结果）
 - 计算性能提高的比例

作业提交内容：

- 一个文本文件，对上述信息给出一个说明
- 两个附件：修改后的 `wordst_good.cpp`，测试用的文本文件。

截止时间：

- 作业发布之后一周内，将上述两个文件提交到自己的GitLab账号所建的小作业3项目中。